Экспериментальная проверка трёх правил do-calculus

Методическая записка: дизайн эксперимента, интервенционные сравнения и обзор программных библиотек

Цель. Описать воспроизводимый способ проверить каждое правило do-calculus на синтетических причинных моделях, а затем перенести его на временную модель Лотки—Вольтерры. Также сравнить библиотеки, которые помогают задавать интервенции, вычислять распределения и проверять причинную идентификацию.

1. Что именно необходимо проверить

Каждое правило do-calculus содержит (а) графическое условие — d-разделение в специально преобразованном графе — и (б) равенство вероятностных выражений. Поэтому корректная проверка должна включать обе части. Одного статистического теста KS недостаточно: он не проверяет графическое условие и не доказывает равенство распределений.

Обозначения: X — переменные, на которые уже выполнена интервенция; Y — целевая переменная; Z — переменные, наблюдение или интервенция над которыми меняется; W — дополнительные условия/контекст. Наборы переменных предполагаются непересекающимися.

2. Три правила и соответствующие сравнения

Правило 1 — вставка/удаление наблюдений

Графическое условие: (Y ⫫ Z | X, W) в графе G с удалёнными входящими стрелками в X ($G_{\bar{X}}$).

> **Равенство: P(Y | do(X), Z, W) = P(Y | do(X), W)**

Эксперимент: выполнить одну и ту же интервенцию do(X) в обеих ветвях. В первой ветви дополнительно отбирать/условливать наблюдения по Z и W; во второй — только по W. Сравнить условные распределения Y при одинаковых значениях/стратах W. Условие по Z должно быть единственным различием между ветвями.

Положительный сценарий: построить DAG, в котором после удаления входящих стрелок в X переменная Z d-разделена с Y при условии X,W. Отрицательный контроль: изменить граф так, чтобы открылся активный путь Z—Y при заданном условии; выбрать параметры, при которых это действительно меняет распределение Y.

Правило 2 — обмен действия и наблюдения

Графическое условие: (Y ⫫ Z | X, W) в графе G с удалёнными входящими стрелками в X и исходящими стрелками из Z ($G_{\bar{X},\underline{Z}}$).

> **Равенство: P(Y | do(X), do(Z), W) = P(Y | do(X), Z, W)**

Эксперимент: сравнить две ветви. В первой задать Z принудительно (do(Z)); во второй не вмешиваться в Z, а наблюдать естественно возникшие значения Z и условливать на них. В обеих ветвях выполнить одинаковую do(X) и сопоставить распределения Y при одинаковых W и значениях Z. Это ключевое правило для проверки различия между «установить Z» и «увидеть Z».

Положительный сценарий: после удаления входящих стрелок в X и исходящих стрелок из Z условие d-разделения выполняется. Отрицательный контроль: построить граф с активным путём между Z и Y в модифицированном графе и проверить, что замена do(Z) наблюдением Z приводит к обнаружимому различию.

Правило 3 — вставка/удаление действий

Графическое условие: (Y ⫫ Z | X, W) в графе G с удалёнными входящими стрелками в X и входящими стрелками в те узлы Z, которые не являются предками W в G с удалёнными входящими стрелками в X ($G_{\bar{X},\underline{Z(W)}}$).

Равенство: P(Y | do(X), do(Z), W) = P(Y | do(X), W)

Эксперимент: сравнить две ветви: do(X) вместе с do(Z) против только do(X). В обеих ветвях одинаково учитывать W. Важно: для графического преобразования правила 3 нельзя механически удалять входящие стрелки во все Z — удаляются входящие стрелки только в те узлы Z, которые не являются предками W в G с удалёнными входящими стрелками в X.

Положительный сценарий: подобрать DAG, в котором условие d-разделения выполняется именно в графе $G_{\bar{X},\underline{Z(W)}}$. Отрицательный контроль: подобрать граф и параметры, где интервенция do(Z) влияет на Y при фиксированном W, то есть удаление do(Z) меняет распределение Y.

3. Общий протокол эксперимента

• Шаг 1. Задать причинный DAG. Сначала использовать небольшие синтетические DAG с известной структурой. Явно определить X, Y, Z, W и структурные уравнения (SCM), включая шумы.

• Шаг 2. Построить модифицированный граф. Для каждого правила выполнить только предписанные операции над стрелками. Проверить d-разделение алгоритмически или вручную.

• Шаг 3. Сформировать пару сценариев. Для каждого правила создать положительный пример, в котором условие d-разделения выполняется, и отрицательный контроль, в котором оно нарушается.

• Шаг 4. Сгенерировать данные. Использовать один и тот же SCM и одинаковые распределения экзогенных шумов в сравниваемых ветвях. Задать интервенции как замену структурного уравнения интервенционной переменной.

• Шаг 5. Получить распределения обеих сторон равенства. Генерировать независимые выборки из нужных интервенционных/условных распределений. При условливании использовать одинаковые страты W (и Z там, где это требуется), сопоставимые диапазоны и одинаковые размеры выборок.

• Шаг 6. Сравнить распределения. Для одномерного Y использовать двухвыборочный KS: статистика D и p-value. Для многомерного Y применять заранее выбранный многомерный тест или сравнивать компоненты отдельно с поправкой на множественные проверки.

• Шаг 7. Оценить не только значимость. Неотклонение нулевой гипотезы KS не доказывает равенство. Заранее задать допустимую границу различия ε и дополнить анализ доверительными интервалами/тестом эквивалентности, размером эффекта и анализом мощности.

• Шаг 8. Повторить эксперимент. Провести несколько независимых прогонов/seed, проверить устойчивость к размеру выборки, шуму и параметрам модели. Зафиксировать код, seed и версии библиотек.

4. Таблицы результатов

Рекомендуется подготовить три отдельные таблицы — по одной на каждое правило. Минимальные столбцы:

Поле	Что фиксировать
Правило / сценарий	Rule 1/2/3; положительный или отрицательный контроль
Графическое условие	Модифицированный граф и результат d-разделения
Сравниваемые выражения	Левая и правая стороны равенства
Настройки генерации	Размер выборки, seed, параметры SCM, шум
Статистика	KS D, p-value; при необходимости CI и размер эффекта

Поле	Что фиксировать
Эквивалентность	Граница ε и результат теста эквивалентности
Вывод	Поддерживает ли эксперимент ожидаемое следствие правила

Интерпретация: положительный сценарий должен показывать близость/эквивалентность сравниваемых распределений при достаточной мощности; отрицательный контроль нужен, чтобы убедиться, что процедура способна обнаружить различие, когда условие правила не выполнено. Само по себе нарушение d-разделения не гарантирует различия при любых параметрах — поэтому отрицательный контроль надо проектировать с ненулевым причинным эффектом.

5. Перенос на систему Лотки—Вольтерры

В модели Лотки—Вольтерры есть обратные связи между популяциями. Стандартные правила do-calculus формулируются для причинных графов без циклов (DAG), поэтому нельзя напрямую считать циклическую схему одновременных уравнений готовым DAG. Рекомендуется развернуть систему во времени: X_t, Y_t, Z_t → X_(t+1), Y_(t+1), с явным заданием задержек и шумов. Полученный граф должен быть ацикличным в пределах выбранного временного горизонта.

В ранее обсуждавшемся описании рассматривались три способа включения внешней переменной z:

1) Внутри скобок: $\dot{x}=(\alpha-\beta y+\lambda z)x$; $\dot{y}=(-\gamma+\delta x+\lambda z)y$.
2) Множитель: $\dot{x}=(\alpha-\beta y)xz$; $\dot{y}=(-\gamma+\delta x)yz$.
3) Аддитивный источник: $\dot{x}=(\alpha-\beta y)x+\eta z$; $\dot{y}=(-\gamma+\delta x)y+\theta z$.

Эти три способа включения z — варианты структурной модели, а не три правила do-calculus. Для каждого варианта нужно отдельно построить временной DAG и определить, какие переменные реально интервенируются. z можно задавать как константу, шумовой процесс или синусоидальный сигнал, но в каждом случае надо явно указать, является ли это do(z), внешним экзогенным воздействием или просто наблюдаемой переменной.

KS по распределениям траекторий может пропустить сдвиг фазы/частоты; подгонка параметров может компенсировать воздействие z. Поэтому полезно анализировать не только сами траектории, но и распределения оценок параметров, фазовые/частотные характеристики и несколько временных срезов. Нулевое воздействие z служит контрольным условием.

6. Библиотеки причинно-следственного анализа

Библиотека	Для чего полезна	Интервенции и ограничения
pgmpy (Python)	Байесовские сети, DAG, причинная идентификация и вероятностный вывод.	CausalInference.query поддерживает do= и evidence= для вычисления P(Y | do(X), Z). Хороша для явных сравнений распределений на дискретных моделях; для непрерывной ODE-системы Лотки—Вольтерры потребуется собственная SCM-симуляция или аппроксимация.
DoWhy (Python)	Полный workflow: модель → идентификация → оценка → refutation/проверки устойчивости.	Поддерживает стратегии идентификации (backdoor, frontdoor, IV, ID) и оценки эффектов. В экосистеме GCM доступны интервенционные выборки и вмешательства в механизмы; это строительный блок, а не готовая кнопка «проверить каждое из трёх правил» отдельно.
pyAgrum (Python)	Причинные байесовские сети, скрытые переменные, причинная идентификация.	causalImpact() вычисляет эффект do-интервенции; документация описывает последовательность: d-separation, backdoor, frontdoor и общий ID algorithm. Полезна как независимый идентификатор/сравнение результата, но не следует приравнивать её к отдельному пошаговому тесту каждого правила.

Библиотека	Для чего полезна	Интервенции и ограничения
Ananke (Python)	Графические причинные модели, идентификация и работа со скрытым смешением.	Полезна как эталон для проверки идентифицирующих выражений и вычисления причинных запросов в интервенционных распределениях. Подходит для контрольных экспериментов на известных графах.
Tetrad / Py-Tetrad	Платформа для причинного обнаружения, графического анализа, тестов независимости и симуляций.	Полезна для построения/изучения DAG, алгоритмов PC/FCI/GES и проверки независимостей. Не стоит считать, что она автоматически реализует полный экспериментальный протокол всех трёх правил.
causal-learn (Python)	Алгоритмы причинного обнаружения: PC, FCI, GES и родственные методы.	Полезна для восстановления структуры и тестов условной независимости. Это прежде всего discovery-инструмент; генерацию интервенционных распределений и сравнение обеих сторон do-calculus обычно нужно реализовать отдельно.
Tigramite (Python)	Причинное обнаружение во временных рядах, включая PCMCI/PCMCI+.	Полезна для анализа лаговых связей и построения временных причинных графов, что особенно актуально для Лотки—Вольтерры. Не является готовым модулем проверки трёх правил Pearl.
dosearch (R)	Поиск идентифицирующих выражений в причинных графах.	Алгоритмически использует правила do-calculus для поиска выражений. Полезен для формальной части идентификации, но не заменяет симуляцию и эмпирическое сравнение распределений.

7. Практическая рекомендация по стеку

Базовый вариант: NetworkX (или аналог) для представления DAG и удаления нужных стрелок; собственная функция проверки d-разделения; DoWhy GCM или собственная SCM для генерации наблюдательных и интервенционных выборок; SciPy для KS; отдельная процедура эквивалентности/доверительных интервалов. pgmpy удобно использовать для проверки распределений на небольших дискретных эталонных моделях.

Независимая проверка: pyAgrum и/или Ananke можно использовать как дополнительные инструменты для проверки идентифицируемости и сравнения получаемых причинных выражений. Tetrad и Tigramite полезны на этапе графов и временной структуры.

Главный вывод: среди рассмотренных библиотек нет оснований полагаться на одну универсальную готовую функцию, которая сама по себе полностью проводит экспериментальную проверку всех трёх правил — от правильного преобразования графа до статистического сравнения обеих сторон равенства. Надёжнее реализовать единый rule-aware тестовый модуль: преобразование графа → d-разделение → генерация соответствующих распределений → статистическое сравнение → положительные и отрицательные контроли.

8. Ссылки на документацию

- pgmpy — User Guide: https://pgmpy.org/documentation
- pgmpy — Causal Estimation и do/evidence queries: https://pgmpy.org/guides/causal_estimation.html
- pgmpy — Causal Identification: https://pgmpy.org/guides/causal_identification.html
- DoWhy — causal workflow: https://www.pywhy.org/dowhy/
- DoWhy — interventions: https://www.pywhy.org/dowhy/main/user_guide/causal_tasks/what_if/interventions.html
- pyAgrum — Causal Inference / causalImpact: https://pyagrum.readthedocs.io/en/latest/CausalInference.html
- Ananke — документация: https://ananke.readthedocs.io/
- Tetrad Manual: https://tetrad-manual.readthedocs.io/
- causal-learn: https://causal-learn.readthedocs.io/
- Tigramite: https://github.com/jakobrunge/tigramite
- dosearch (R): https://cran.r-project.org/web/packages/dosearch/vignettes/dosearch.html

Примечание о точности описания библиотек: возможности и API меняются между версиями. Перед реализацией следует сверить актуальную документацию и закрепить версии зависимостей в окружении проекта.